Chair of Network Architectures and Services
Department of Informatics
Technical University of Munich

# Building a Production-Ready Data Pipeline for Large-Scale Data Processing

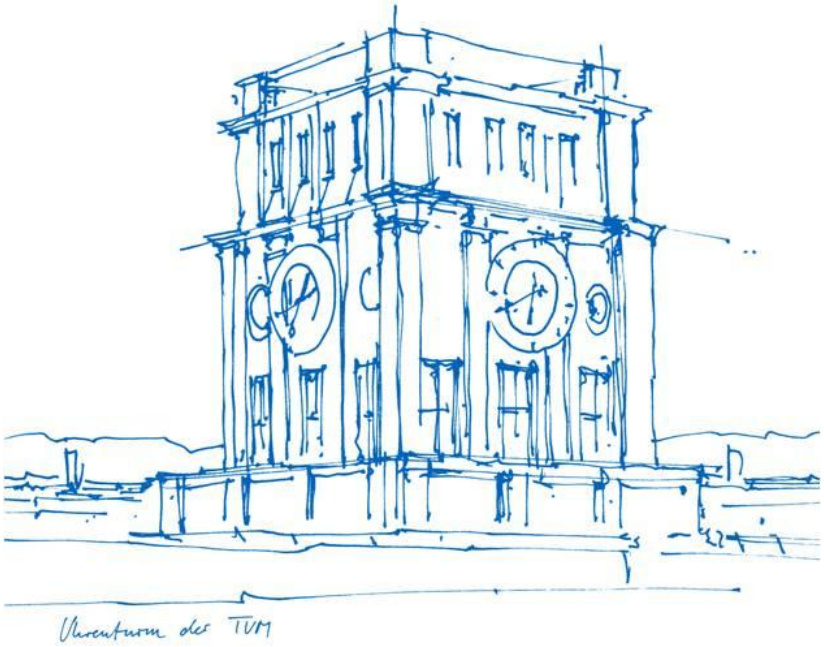
*Final Talk*

M. Furkan Atasoy

Advisor(s): Filip Rezabek, Kilian Glas
Supervisor: Prof. Dr.-Ing. Georg Carle

Technical University of Munich (TUM)
Department of Informatics
Chair of Network Architectures and Services

Garching, 05.06.2024



# Agenda

1. Introduction
2. Analysis
3. Design
4. Implementation
5. Evaluation
6. Conclusion
7. Demo

# Introduction

**Challenge:**

- Managing complex, high-volume blockchain data

**Need:**

- High-quality, accessible research data platform

**Goal:**

- Building a production-ready data pipeline
- Integrating it with a JupyterHub platform
- Preparing use cases of the system

# Use Case: MEV (Maximal Extractable Value) Analysis

**What is MEV:**

- Max profit from reordering transactions in a block
- Done by block creators

**Why we chose MEV:**

- Involves complex data processing
- Real-world relevance

# Analysis: Requirements

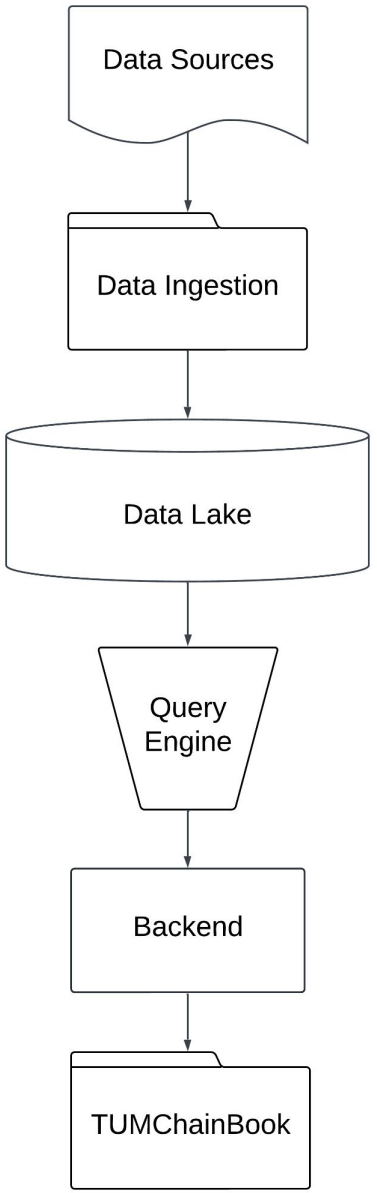
1. Scalability
2. Simplicity and Maintainability
3. Performance Efficiency
4. Extensibility
5. Observability
6. Security

# Design: Architecture (Simplified)

- Process data step by step
- Use Data Lake system
- Query data flexibly
- Access data from TUMChainBook

**Requirement Analysis**

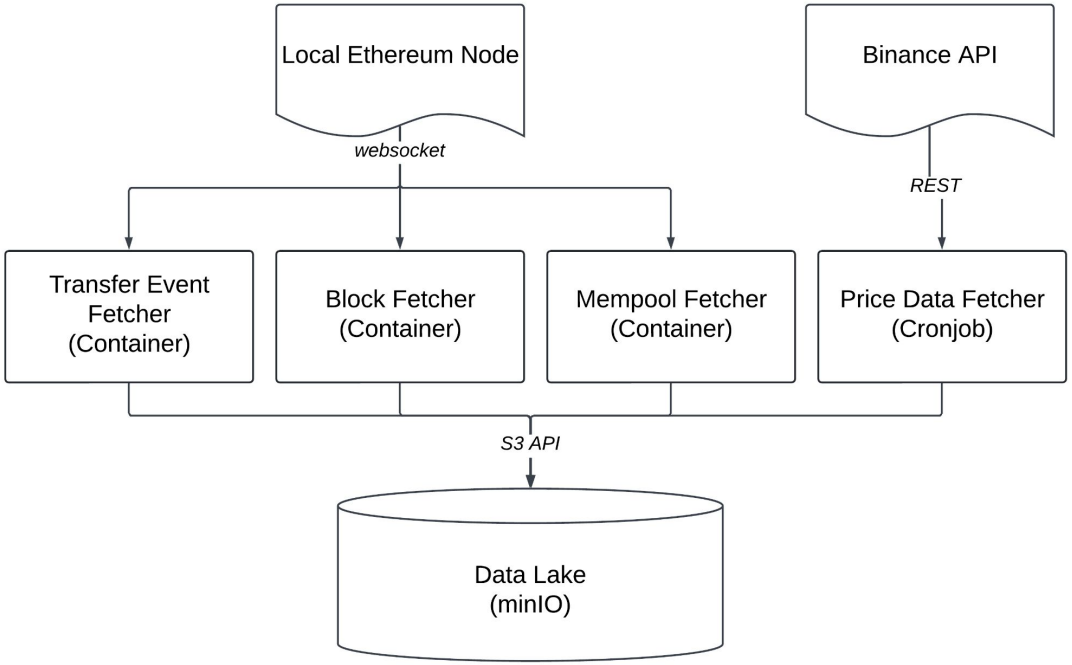
- Simplicity and Maintainability



# Design: Architecture (Collection)

*Use Case: MEV Analysis*

- Ingest continuous data with always-running containers
- Ingest non-continuous data with cronjobs in regular intervals
- Store raw data with Parquet format in minIO




**Requirement Analysis**

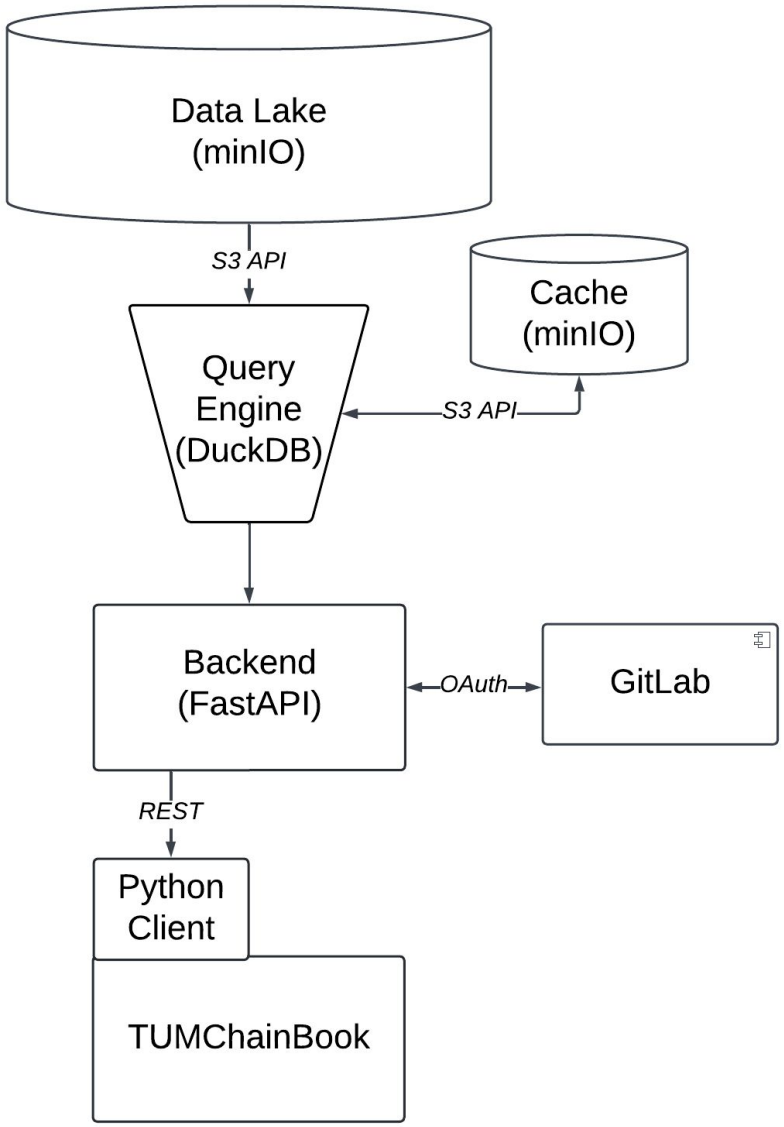
- Scalability
- Extensibility

# Design: Architecture (Access)

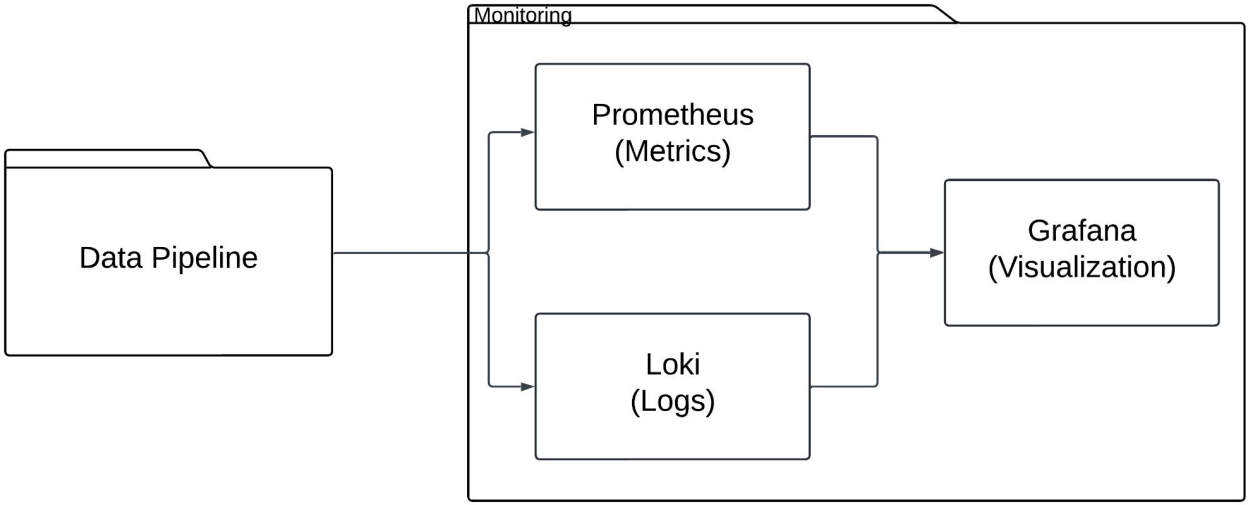
- Query data via DuckDB from Data Lake
- Serve queried data via Backend
- Get data in TUMChainBook
- Cache data on-demand and serve through it

**Requirement Analysis**

- Performance Efficiency
- Security



# Design: Architecture (Monitoring)

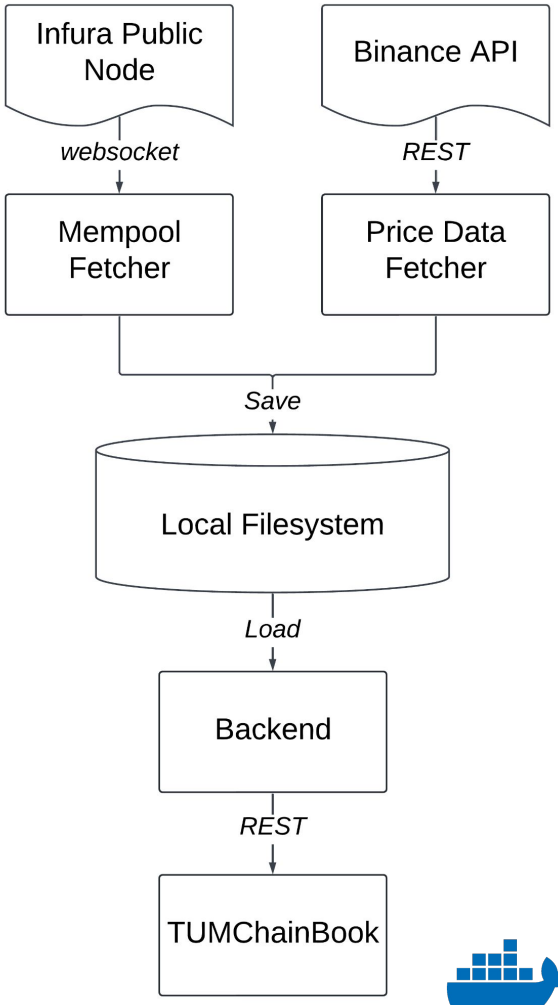


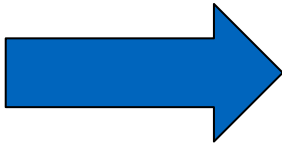

- Collect Metrics via Prometheus
- Collect Logs via Loki
- Visualize Metrics and Logs via Grafana

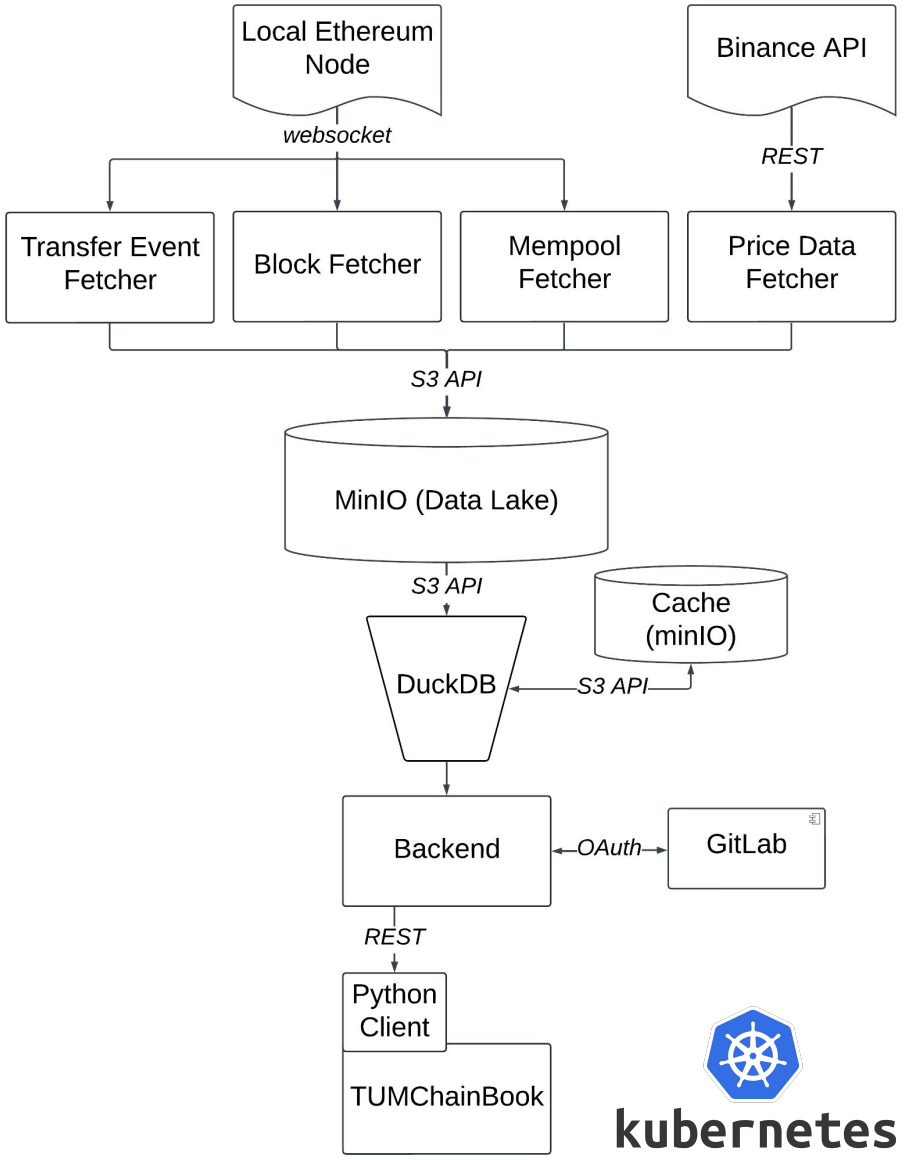
**Requirement Analysis**

- Observability
- Maintainability

# Implementation: Iterative Development





# Evaluation: Metrics

**Ingestion Metrics:**

- Data Growth
- Ingestion Latency
- Ingestion Throughput

**Query Metrics:**

- Query Latency
- Query Throughput

**Resource Usage Metrics:**

- CPU Usage
- Memory Usage

# Evaluation: Workload Emulation Setup

## Configuration

- 60 users
- Running Jupyter notebooks

## User Traffic

- Exponential inter-arrival times
- Open system benchmarking

LISTING 7.1: Calculation of Sleep Time

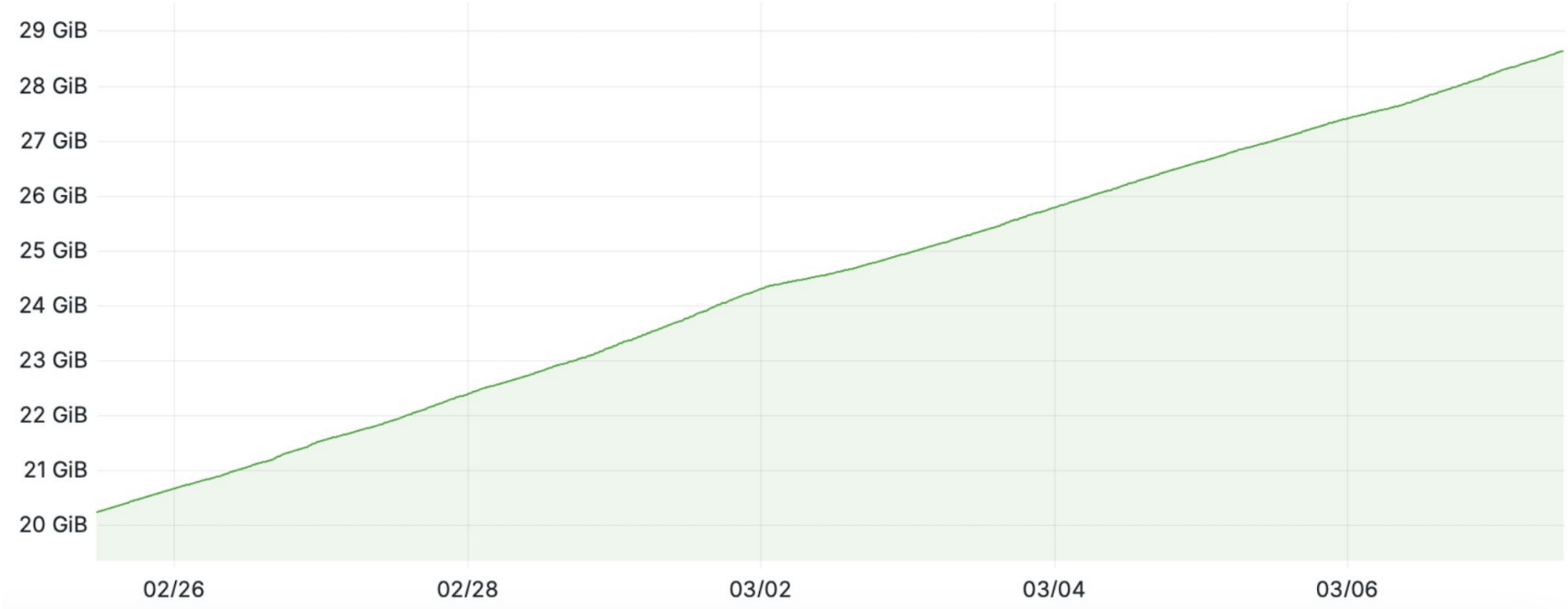
```
1 import random
2 mean_interarrival_time = 3600 # in seconds
3 sleep_time = random.expovariate(1.0 / mean_interarrival_time)
```

## User Personas

- Trade Researchers (30 users)
- Validation Researchers (30 users)

# Evaluation: First Results

## Data Growth

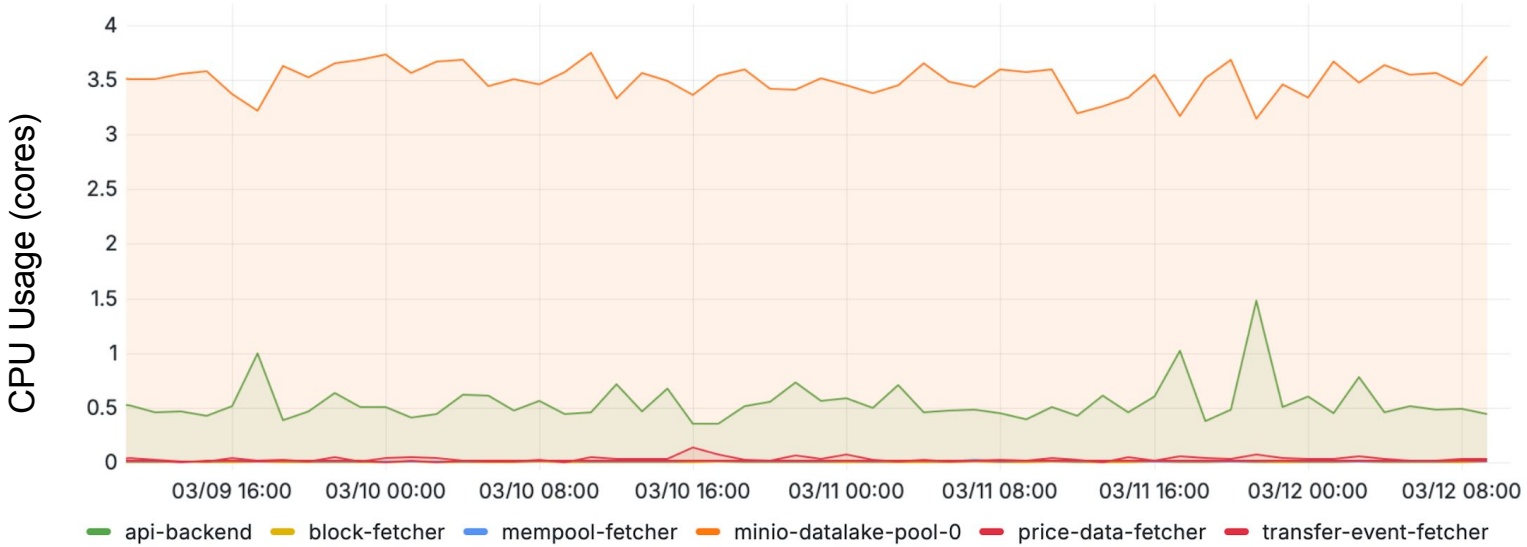



$$r_{\text{daily}} = \frac{8.4 \text{ GB}}{10 \text{ days}} = 0.84 \text{ GB/day} \tag{7.1}$$

$$S_{\text{future}} = 28.6 \text{ GB} + (0.84 \times 730) \text{ GB} = 641.8 \text{ GB}$$

# Evaluation: First Results

## CPU and Memory Usage




| Service | CPU Usage (cores) | | Memory Usage (MiB) | |
|---|---|---|---|---|
| | Mean | Max | Mean | Max |
| MinIO Datalake | 3.51 | 3.78 | 18,000 | 18,700 |
| API Backend | 0.54 | 1.34 | 5,410 | 8,010 |
| Transfer Event Fetcher | 0.032 | 0.22 | 78.2 | 80.4 |
| Price Data Fetcher | 0.015 | 0.018 | 71.6 | 73.3 |
| Mempool Fetcher | 0.012 | 0.029 | 81.9 | 88.8 |
| Block Fetcher | 0.0016 | 0.0081 | 103.7 | 119 |

# Evaluation: Performance Improvements

**Reducing Load on Bottlenecks**

- Caching Layer
  - Run on the fast SSD storage

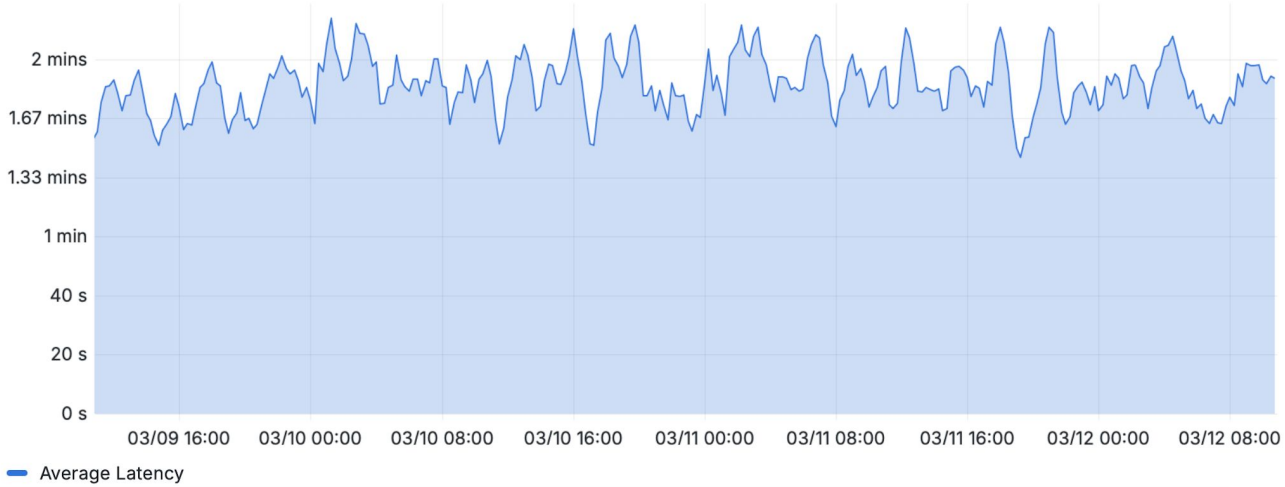
**Preventing Resource Overload**

- Defined Resource Limits
- Enforced Query timeouts

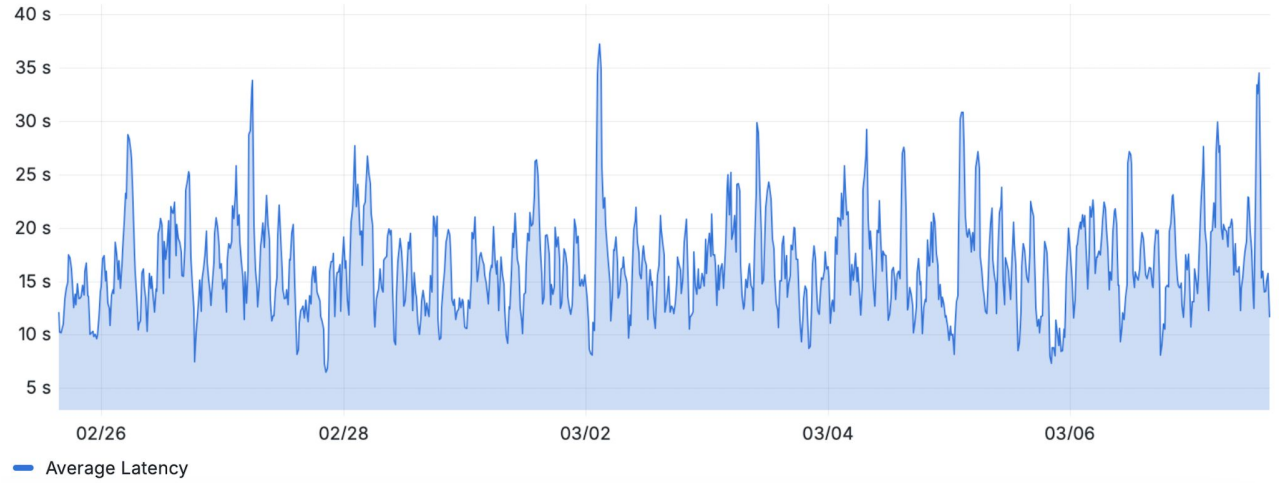
# Evaluation: Comparison

## Impact of Caching: Query Latency

*without Caching*



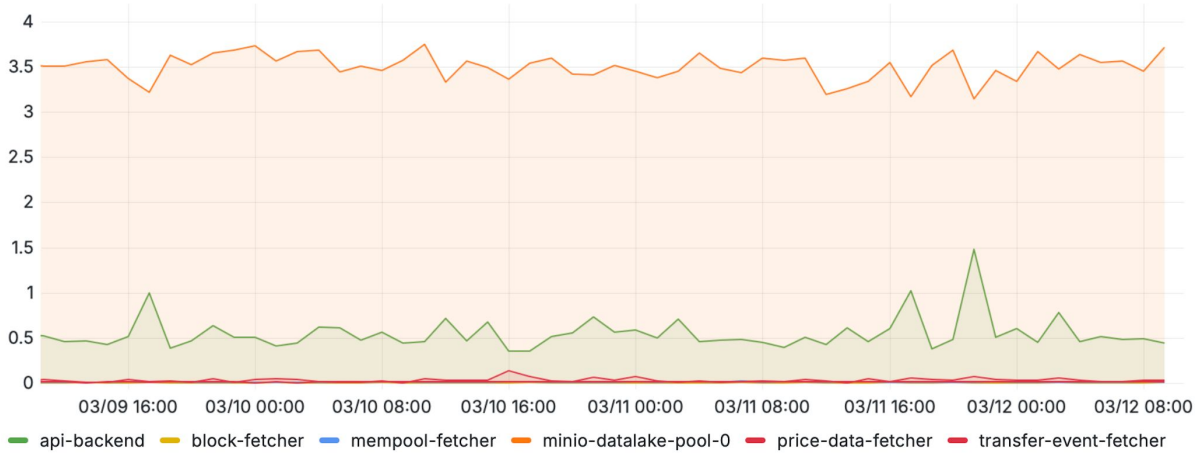

*with Caching*



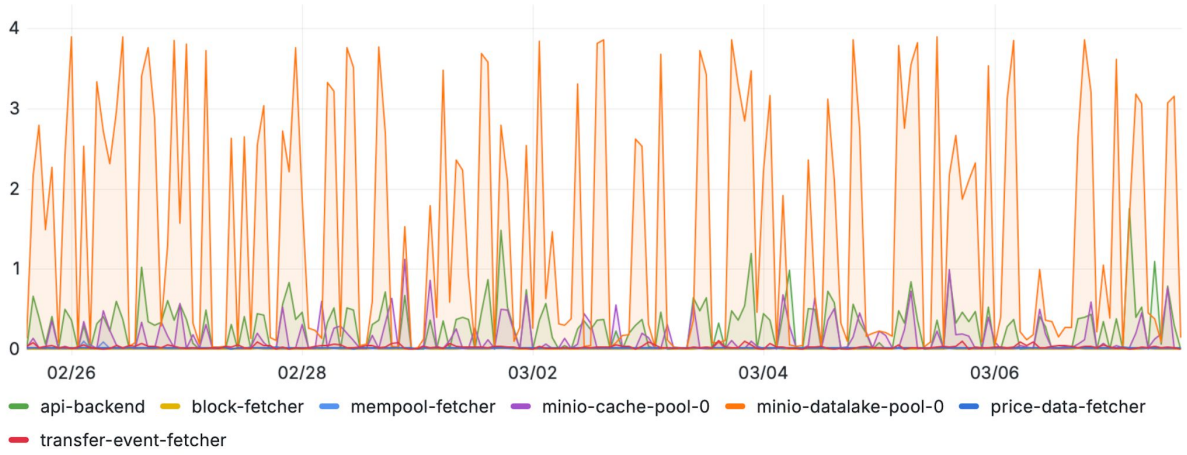
# Evaluation: Comparison

## Impact of Caching: Resource Consumption

*without Caching*




*with Caching*



# Conclusion: Research Questions

**RQ1:** *How can we design and deploy a pipeline that can continuously ingest blockchain data over long periods?*

- Meet non-functional requirements

**RQ2:** *Which established technologies and solutions should be used to make the pipeline robust and production-ready?*

- MinIO, DuckDB, JupyterHub, Kubernetes

**RQ3:** *What is the most suitable architecture for building a large-scale data collection pipeline?*

- ELT architecture with caching layer

**RQ4:** *Which representative use cases are suitable for demonstrating the effectiveness of the pipeline?*

- Maximal Extractable Value (MEV) Analysis

# Demo

# Thank you for listening!

# Appendix - Introduction: Research Questions

**RQ1:** *How can we design and deploy a pipeline that can continuously ingest blockchain data over long periods?*

**RQ2:** *Which established technologies and solutions should be used to make the pipeline robust and production-ready?*

**RQ3:** *What is the most suitable architecture for building a large-scale data collection pipeline?*

**RQ4:** *Which representative use cases are suitable for demonstrating the effectiveness of the pipeline?*

# Appendix - Related Works

1. Ilteber's Thesis – Data Collection Pipeline for Blockchain Platforms

2. TUMChainBook

3. Dune Analytics

# Appendix - Implementation: Node Deployments

**Frodoi**

- 8TB SSD storage, high-computing power
- Run local Ethereum node: Erigon

**Pippini**

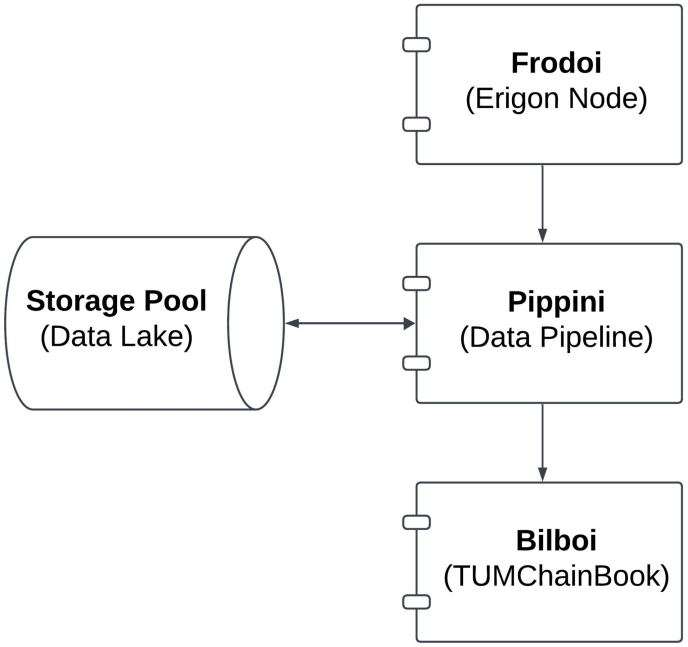
- 8TB SSD storage, high-computing power
- Run data pipeline

**Storage Pool**

- 40TB HDD Storage, low-computing power
- Run Data Lake (minIO physical storage)

**Bilboi**

- 200GB SSD storage, high-computing power
- Run TUMChainBook



# Appendix - Evaluation: Workload Emulation Setup

## User Traffic

- Exponential inter-arrival times
- Open system benchmarking

LISTING 7.1: Calculation of Sleep Time

```
1 import random
2 mean_interarrival_time = 3600 # in seconds
3 sleep_time = random.expovariate(1.0 / mean_interarrival_time)
```

## User Personas

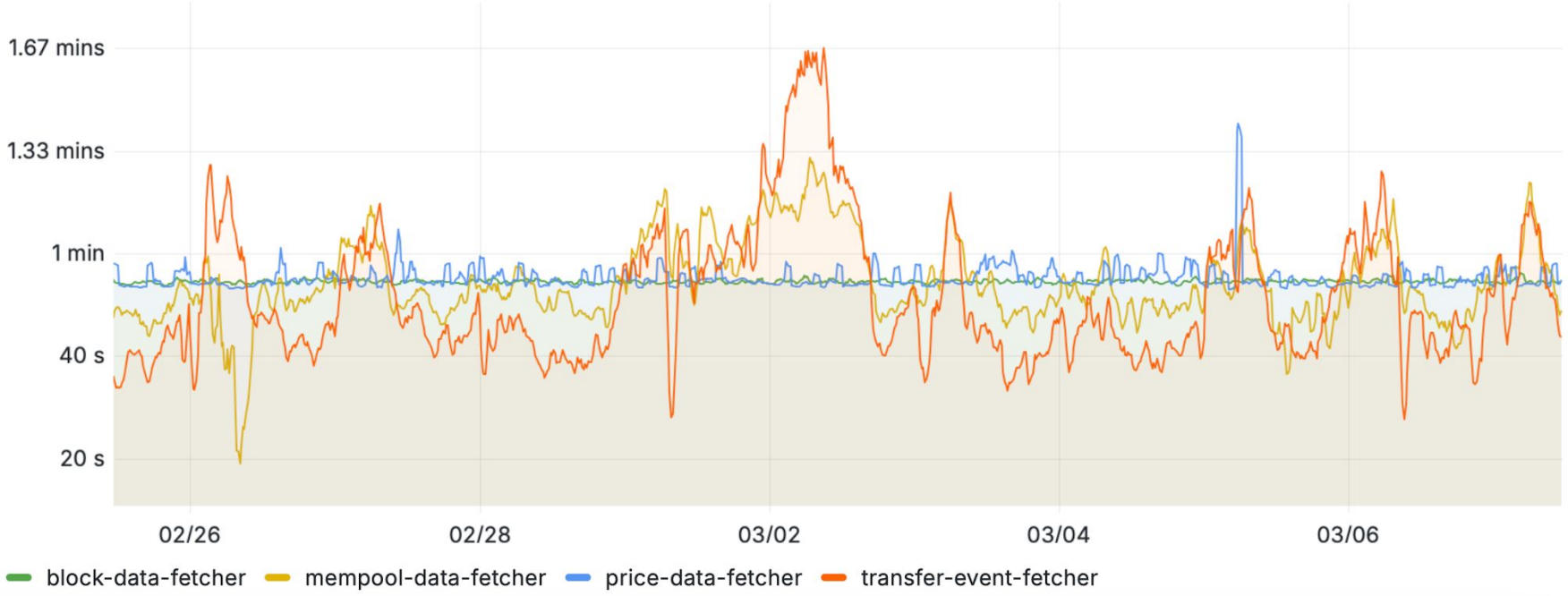
- Trade Researchers (30 users)
- Validation Researchers (30 users)

## Cache Behavior

- Continuous data ingestion
- Cache usage probability

# Appendix - Evaluation: First Results

## Ingestion Latency



## Impact of Caching: Resource Consumption

| Service | Without Cache (CPU Cores) | | With Cache (CPU Cores) | |
|---|---|---|---|---|
| | Mean | Max | Mean | Max |
| MinIO Datalake | 3.51 | 3.78 | 1.47 | 3.89 |
| API Backend | 0.54 | 1.34 | 0.29 | 1.83 |
| MinIO Cache | - | - | 0.15 | 1.12 |
| **Total** | **4.05** | - | **1.91** | - |

# Appendix - Conclusion: Future Work

1. Expanding Data Sources

2. Metadata Management and Improved Data Exploration

3. Dynamic Caching Strategies